Tasky V2 Requirements

This requirements document explains the main functionality behind Tasky V2 with responsive UI design and dark mode support. The exact looks and colors of specific UI elements can be seen on the updated mockups.

This serves as an overall impression of what the app should be able to do. Feel free to decide on your own how you implement specific things (e.g. loading progress indicators or error message formatting).

Updated mockups can be found here: https://www.figma.com/design/nblGPLxYBWvuZwyLfWwpzQ/Tasky?node-id=0-1&p=f&t=nUMc7Ymqp3qVEc8k-0

Icons

All icons for the app can be Material design icons or taken from the mockups as SVG (in case an equivalent Material icon doesn't exist)

App Theme

The app supports both light and dark themes with full responsive UI design. Users should be able to toggle between themes, with the app respecting system theme preferences by default.

Theme Support:

- Light theme (original design)
- Dark theme (new requirement)
- System theme detection and automatic switching
- Theme preference persistence across app sessions

All theme colors and font styles are described in the design system: https://www.figma.com/design/nblGPLxYBWvuZwyLfWwpzQ/Tasky?node-id=1-4740&p=f&t=7quEQkOHyUHUz44s-0

Responsiveness

The app supports multiple device types and orientations with fully responsive layouts. The follow in an overview and more specific requirements will be given at each screen:

Device Support:

- Mobile devices (phones)
- Tablets
- Portrait and landscape orientations for all devices

Responsive Behavior:

- **Mobile Portrait:** Single-column layouts, standard mobile navigation patterns
- **Mobile Landscape:** Optimized horizontal layouts, potentially side-by-side content where appropriate
- **Tablet Portrait:** Wider layouts taking advantage of additional screen real estate
- **Tablet Landscape:** Multi-column layouts where beneficial, potentially split-screen views for detail screens

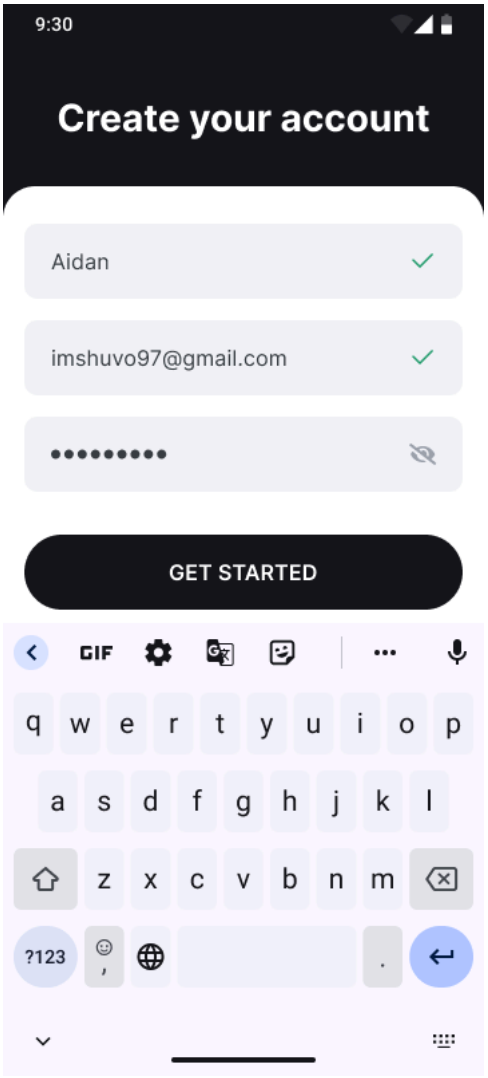
Splash Screen

- The app displays a splash screen during which it checks if the user has an active session or needs to login
- If the user has an active session, navigate to the agenda screen, else to login screen
- Responsive design ensures proper display across all device sizes and orientations

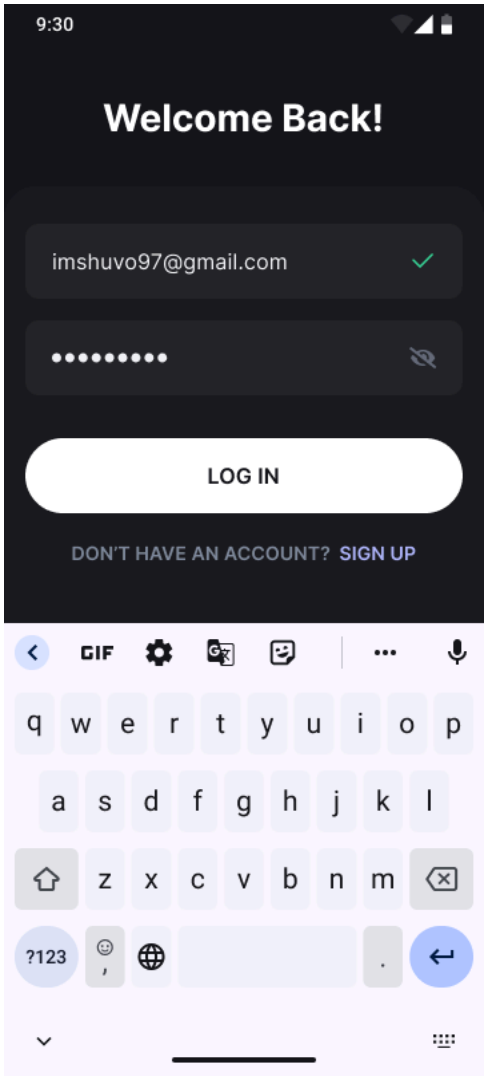
Splash Screen - Portrait

Splash Screen - Landscape

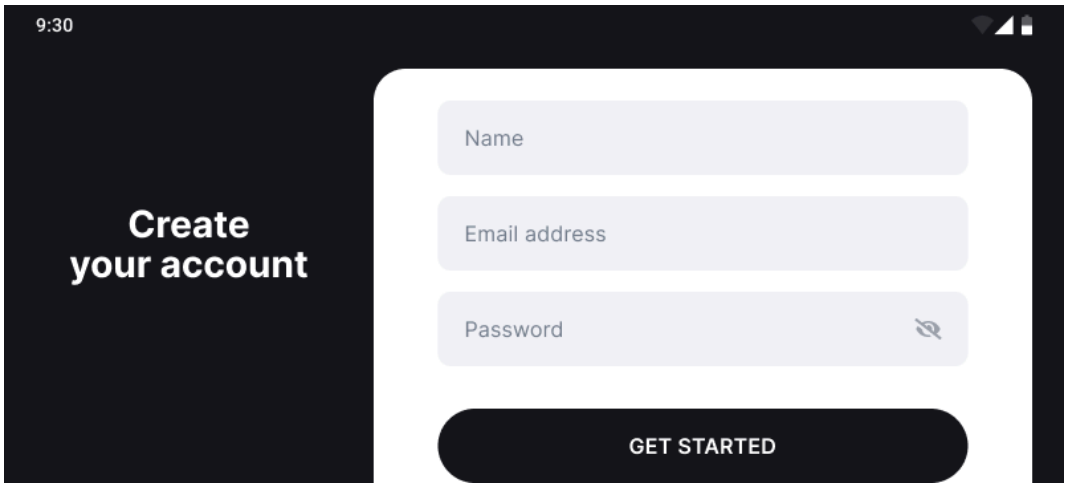
Authentication



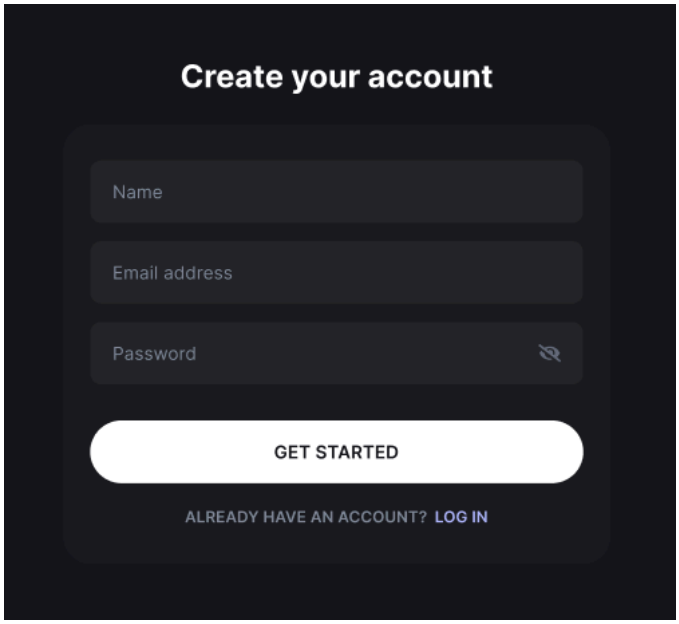
Register Screen - Light - Portrait



Login Screen - Dark - Portrait



Register Screen - Light - Landscape



Register Screen - Dark - Tablet

Registration and Login Screens:

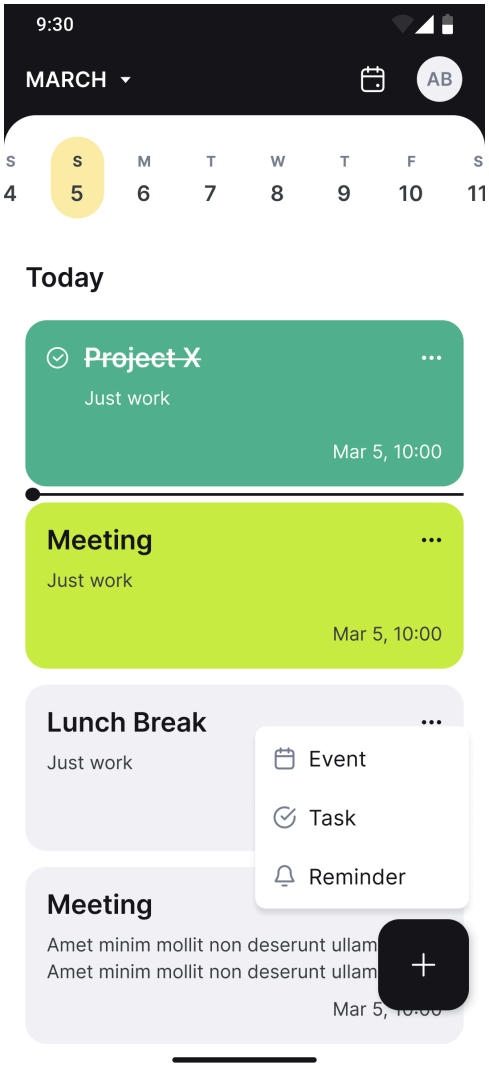
- Users can register and login with responsive form layouts
- Registration requires: full name, email address, and password
- Full name must be between 4 and 50 characters
- Email must be valid format
- Password must contain lowercase and uppercase letters plus at least one digit

- Password must be at least 9 characters long
- Valid field entries show checkmarks on the right side
- Password fields include visibility toggle with eye icon
- Successfully logged-in users remain logged in while token is valid

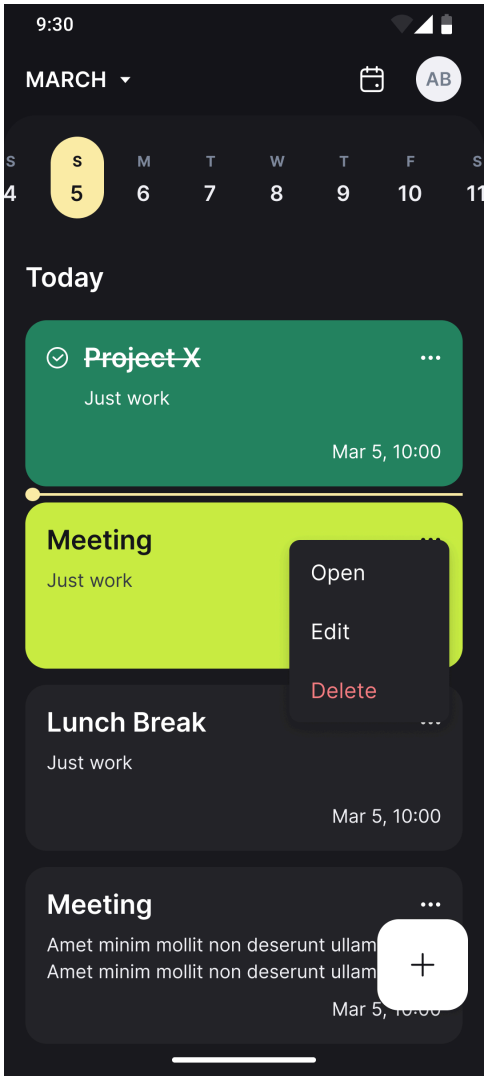
Responsive Design:

- **Mobile Portrait:** Stacked form elements with full-width inputs
- **Mobile Landscape:** Compact form layout optimized for horizontal space
- **Tablet:** Centered form with appropriate sizing, potentially with additional spacing

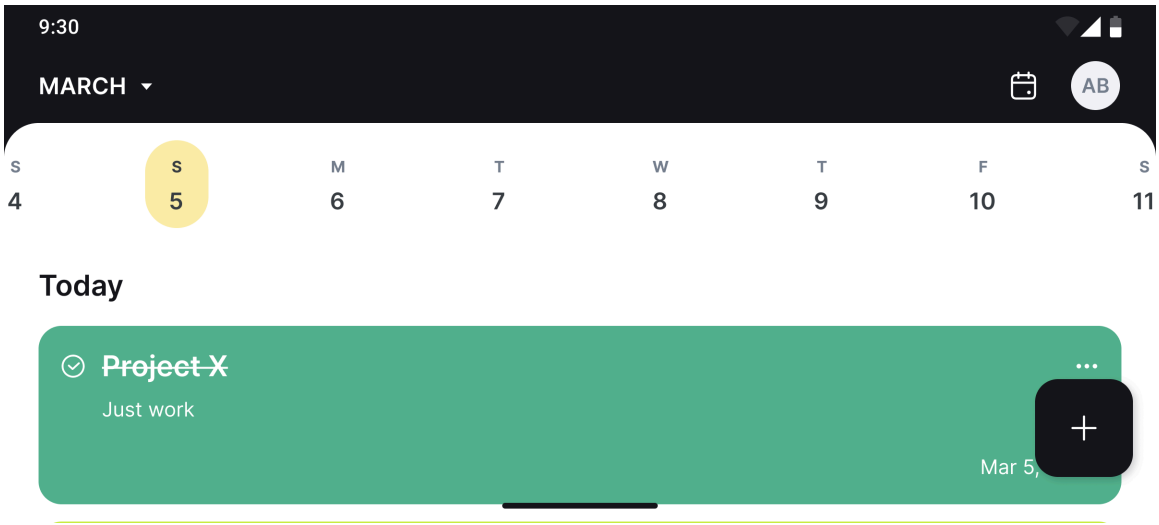
Agenda Screen



Agenda Screen - Light - Portrait



Agenda Screen - Dark - Portrait



Agenda Screen - Light - Landscape

The agenda screen serves as the main interface after login, with enhanced responsive design:

Core Functionality:

- Displays user's agenda for a given day (events, tasks, reminders)
- Events are light green, tasks are dark green, reminders are grey (*Light Mode)*
- All agenda items ordered ascending by time
- Attendee chips shown for events where user is attendee
- Task completion toggle via circle tap
- Time needle shows current time position between agenda items
- Live time needle updates (it only considers event start times)
    - The needle never overlaps the agenda items, it can only be either above or below an agenda item in the list

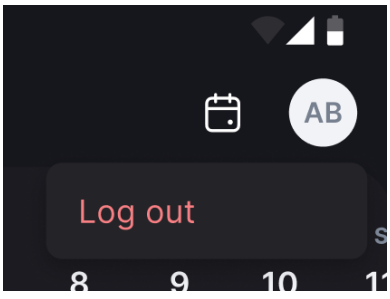
List items:

- Each agenda item should display its title, description and time (For events, display both `from` and `to` times)
- Each agenda item should have a context drop down menu to open, edit or delete it

- A click on open is equivalent to clicking on the agenda item. It will simply open it on the detail screen in non-edit mode
- A click on edit will open the item on the detail screen in edit mode
- A click on delete should show a confirmation dialog with the options `Cancel` and `OK` to let the user confirm the deletion

Navigation and Controls:

- Toolbar with date selection (Material3 date picker)
- Both month text and calendar icon open date picker
- Horizontal scrollable date row showing the past and next 15 days from today
  - If today is June 1st then the row's date range would be May 17th up to June 16th
  - June 1st would be the preselected date and focused in the middle
- Profile icon with user initials and logout dropdown
  - E.g. if the full name is `Philipp Lackner`, display the initials → PL
  - If the full name consists of one word, display the first 2 characters → PH
  - If there is a middle name, display the initials of the first and last name, for `First Middle Last`, it would therefore be `FL`



Logout Dropdown - Dark

👉 Log out functionality is only available when the user is online

- Floating action button for adding new agenda items

    - Clicking on it will open a drop down with each agenda item type being one option

Responsive Layouts:

- Additional padding where needed to space out elements

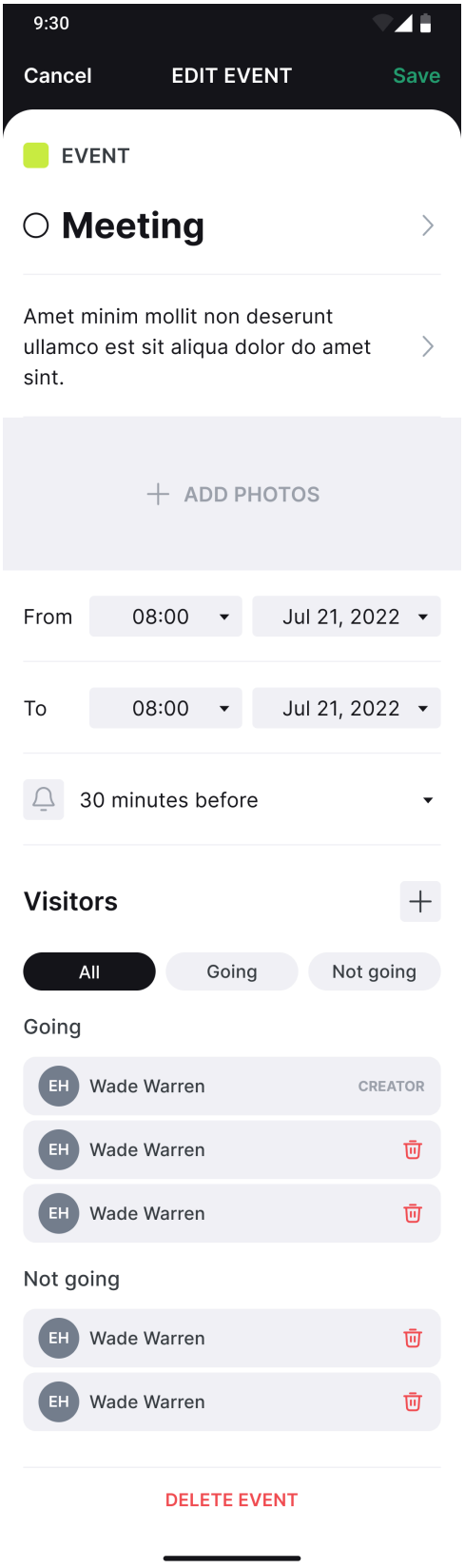
Context Menus:

- Each agenda item has context menu: Open, Edit, Delete
- Delete requires confirmation dialog
- Edit opens detail screen in edit mode

Theme Adaptations:

- All agenda colors adapt appropriately for dark theme visibility
- Time needle maintains visibility in both themes

Event Detail Screen



Core Data:

- Title, description (optional), time range (from/to)
- Reminder time options:
  - 10 minutes before
  - 30 minutes before
  - 1 hour before
  - 6 hours before
  - 1 day before
- Up to 10 photos (max 1MB each, auto-compression)
- Attendee list

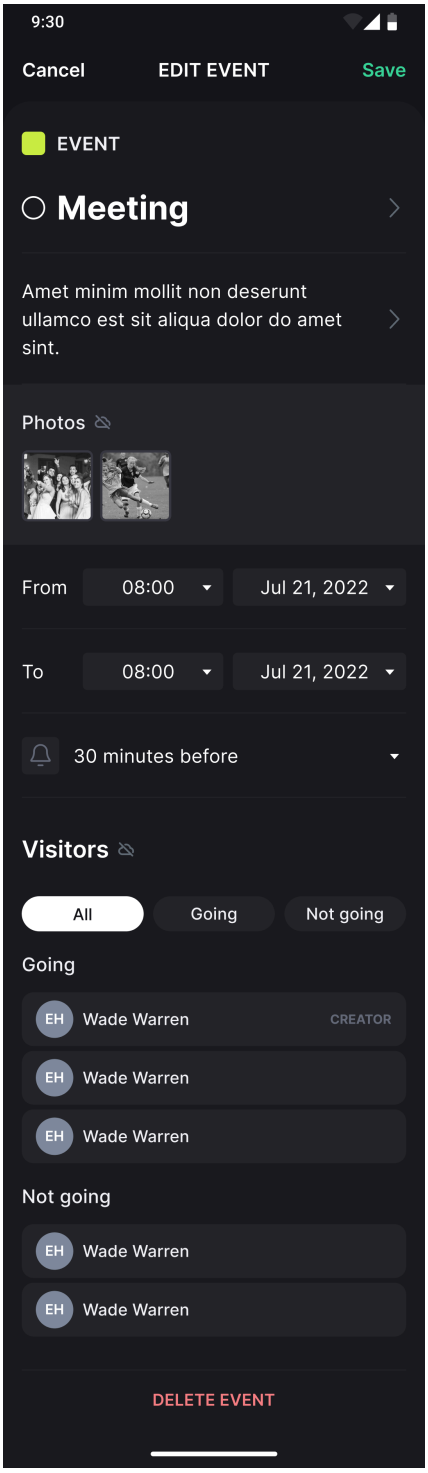
Edit Modes:

- View mode (default for existing events)
- Edit mode (via toolbar edit icon or context menu)
- New events start in edit mode
- Save button replaces edit icon when editing

Photo Management:

- Grid-style photo display with reasonable max dimensions
- Add photo opens file chooser for images
- Photo tap opens Photo Detail Screen with delete option

Event Detail Screen - Edit Mode - Light - Portrait



Event Detail - View Mode - Dark - Portrait

- Compression for photos >1MB, skip if still too large
  - The event should still be successfully created, but without these photos that are too large
  - The user should see a message like `x photos were skipped because they were too large` (can be a simple Toast)
- Offline mode shows offline icon, no photo editing allowed
- Photos should be cached when loaded from remote data source
  - Therefore, caching locally added photos is **not** necessary (caching photos loaded from remote is however)

👉

Photo Upload/Confirmation Flow for Events

The Tasky API implements a **two-phase photo upload system** for events to ensure reliable photo handling and prevent issues with large file uploads. Here's how it works:

Phase 1: Event Creation/Update with Photo Keys

For Creating Events (`POST /event`)

```
{
  // ... other fields
  "photoKeys": ["photo01", "photo02"], // ← Photo keys to upload
}
```

For Updating Events (`PUT /event/{eventId}`)

```
{
  // ... other fields
  "newPhotoKeys": ["photo03", "photo04"], // ← New photos to add
  "deletedPhotoKeys": ["unique-id"], // ← Existing photos to remove
}
```

Server Response

Both endpoints return the same response structure:

```
{
  "event": {
    "photoKeys": [...], // ← Photos are NOT yet accessible
    // ... other event fields
  },
  "uploadUrls": [
    {
      "photoKey": "photo01", // ← Your original key
      // 👇 Server-generated key
```

```
    "uploadKey": "4d0daede-fbe9-449b-ae26-c4337e3515a4",
    "url": "https://s3.amazonaws.com/..." // ← URL to upload to
   },
   // ... other photos to upload
  ]
}
```

Important: At this stage, photos are NOT yet visible or accessible in the event.

Phase 2: Photo Upload & Confirmation

Step 1: Upload Photos to S3

Use the provided upload URLs to upload your photos directly to S3 - no special headers or authentication required, the URL has it baked in already. Note that the URL will contain the server-generated ID received as the `uploadKey`.

Step 2: Confirm Uploads (`POST /event/{eventId}/confirm-upload`)

After successfully uploading all photos, confirm the uploads:

```
{
  "uploadedKeys": [
    // Use the uploadKey from step 1
    "4d0daede-fbe9-449b-ae26-c4337e3515a4",
    "7b2eef45-9cdd-4883-bf37-f5448f4626b5"
  ]
}
```

Final Response

The server validates the uploads and returns the event with accessible photos:

```
{
  "event": {
    "photoKeys": [
      {
        // Server-generated key
        "key": "4d0daede-fbe9-449b-ae26-c4337e3515a4",
        // Download URL generated
        "url": "https://..."
      },
      {
        "key": "7b2eef45-9cdd-4883-bf37-f5448f4626b5",
        "url": "https://..."
      }
    ],
    // ... other event fields
  }
}
```

Attendee Management:

- Add attendee via bottom sheet with email input
  - Same as for the other email fields in the app, display a checkmark next to it if it's a valid email
  - The add button should only be clickable if the email is valid
  - If the user clicks on the button for a valid email format, you still need to check if a user with that email exists - this works via the `/attendee` endpoint
    - If it doesn't exist, show a corresponding error in the bottom sheet
    - Typing something should clear any displayed error message
- Email validation with checkmark indicator
- Check user existence via `/attendee` endpoint
- Filter attendees: All/Going/Not Going

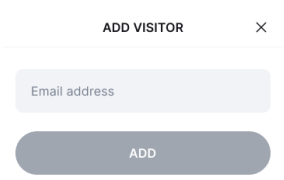
- Creator badge display, removal options for added attendees
- Offline mode shows offline icon, no attendee management
- View as attendee
    - Attendees can’t edit events they’ve been invited to
        - Therefore, the edit and save button in the toolbar should be hidden in this case
        - An attendee can however change their own personal reminder time for this event - the reminder type drop down should always be clickable and editable for an attendee.
        - Furthermore, instead of seeing a delete event button at the bottom, attendees should be able to join or leave the event.
        - If you’ve left an event as an attendee, you should not receive notifications for it.
- Event creators can’t mark themselves as going/not going - they’re always considered as going

> 👉🏽 **Offline mode restrictions:** In offline-mode, the user can’t edit the event’s photos or attendees.

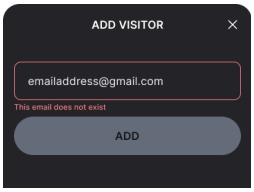
Adding and removing attendees

There’s an important difference between joining/leaving as an attendee and deleting an event as attendee. If you are attendee of an event and you want to leave it, you need to call `PUT /event` and mark yourself as going in the attendee list.

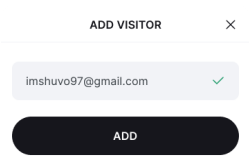
If you want to completely delete an event as an attendee however (so that it doesn’t show up in your agenda anymore), you need to call the `DELETE /attendee` endpoint which will remove the attendee from the list of the event.



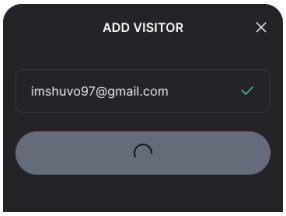
Add Attendee Sheet - Step 1 - Light



Add Attendee Sheet - Step 2 - Dark



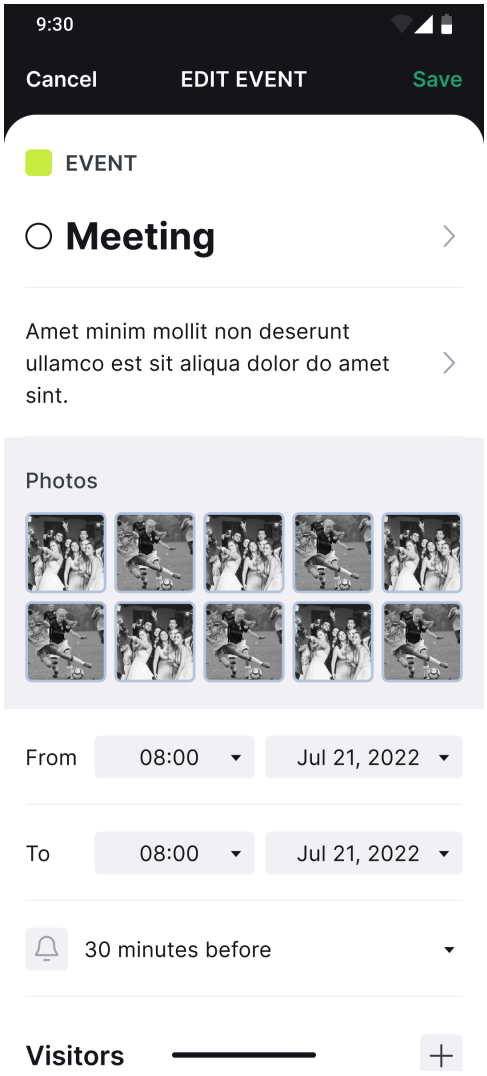
Add Attendee Sheet - Step 3 - Light



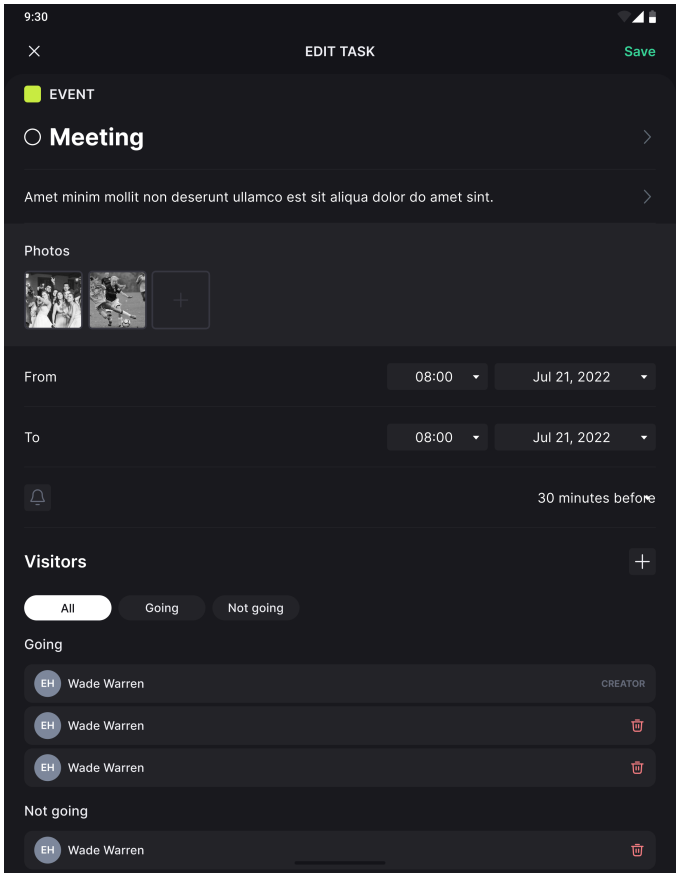
Add Attendee Sheet - Step 4 - Dark

Responsive Design:

- **Mobile Portrait:** Stacked sections, scrollable content with grid layout for photos
- **Mobile Landscape:** Same as portrait with scrollable row for photos instead of a grid layout
- **Tablet:** Photos are also in a scrollable row layout



Event Details - Grid Photos - Light - Portrait

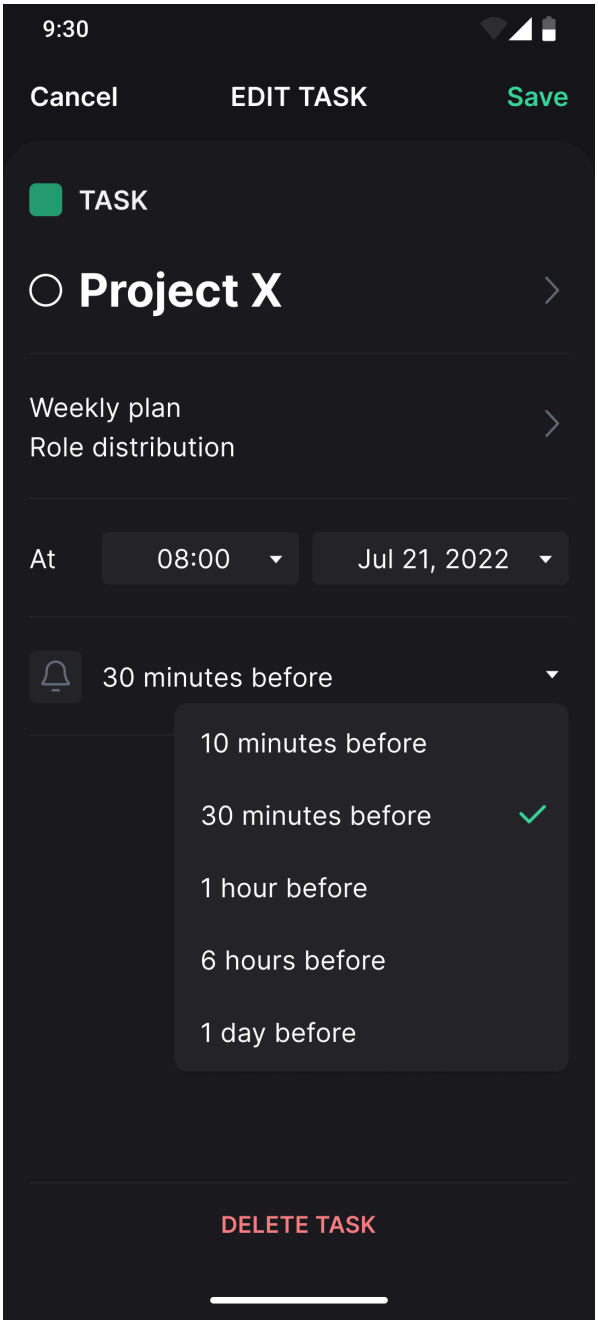


Event Details - Scrollable Row Photos - Dark - Tablet

Role-based Access:

- **Event Creator:** Full edit access, delete event button
- **Attendee:** View only, personal reminder time editable, join/leave options

Task Detail Screen



Task Detail - Edit Mode - Dark - Portrait

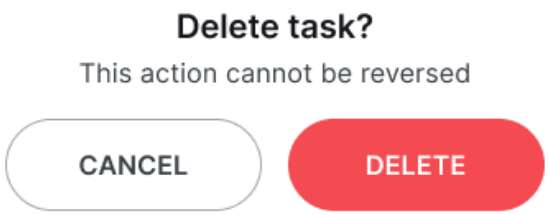
- The task detail screen is used to create, edit and display a task

Core Data:

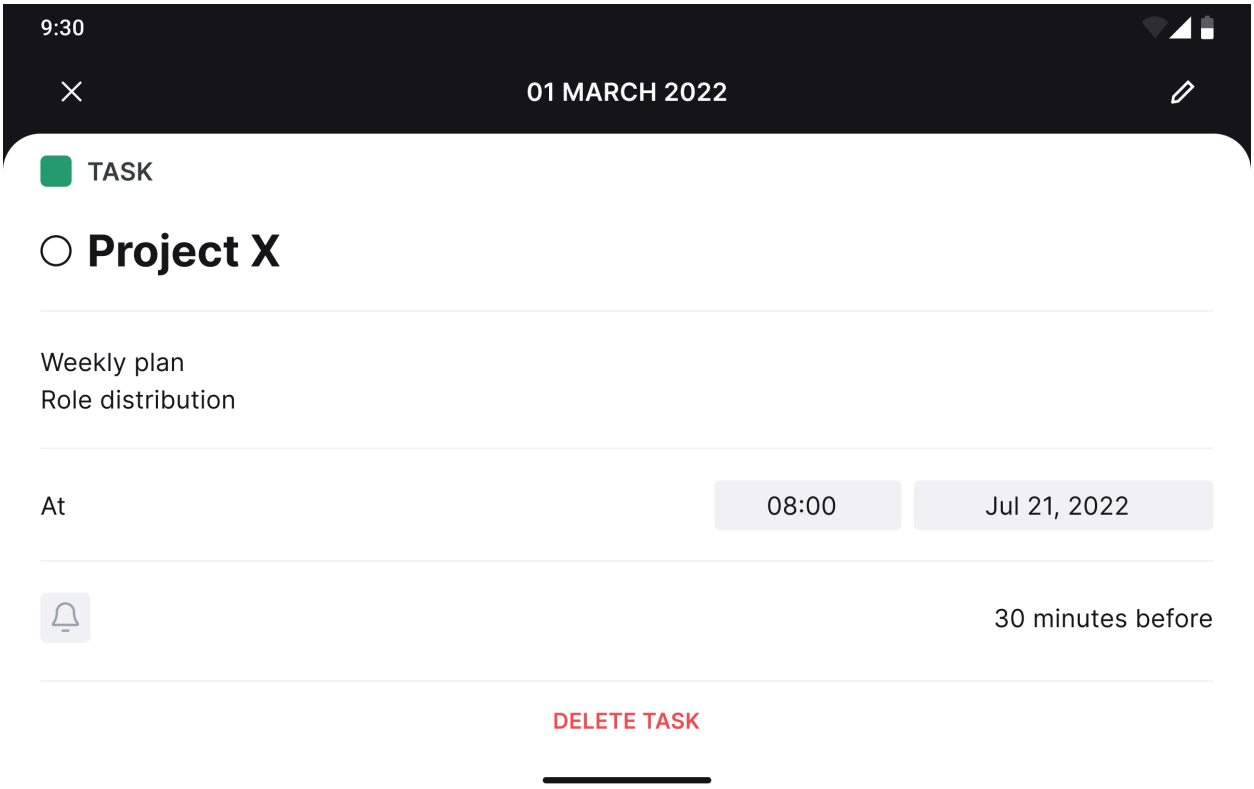
- A task consists of the following data:
  - Title
  - Description
  - Time (Date + time)
  - Reminder time (same options as for events)
  - Completion status
- Cannot mark as done from detail screen (agenda screen only)
- New tasks default to undone

Edit Behavior:

- Same edit mechanism as events (view/edit modes)
- Title/description editing via separate screens
- Date/time selection via system dialogs
- Delete button shows confirmation bottomsheet

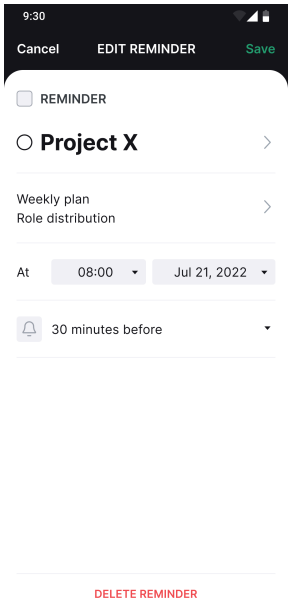


Delete Task Confirmation Bottom Sheet - Light



Task Detail - View Mode - Light - Landscape

Reminder Detail Screen



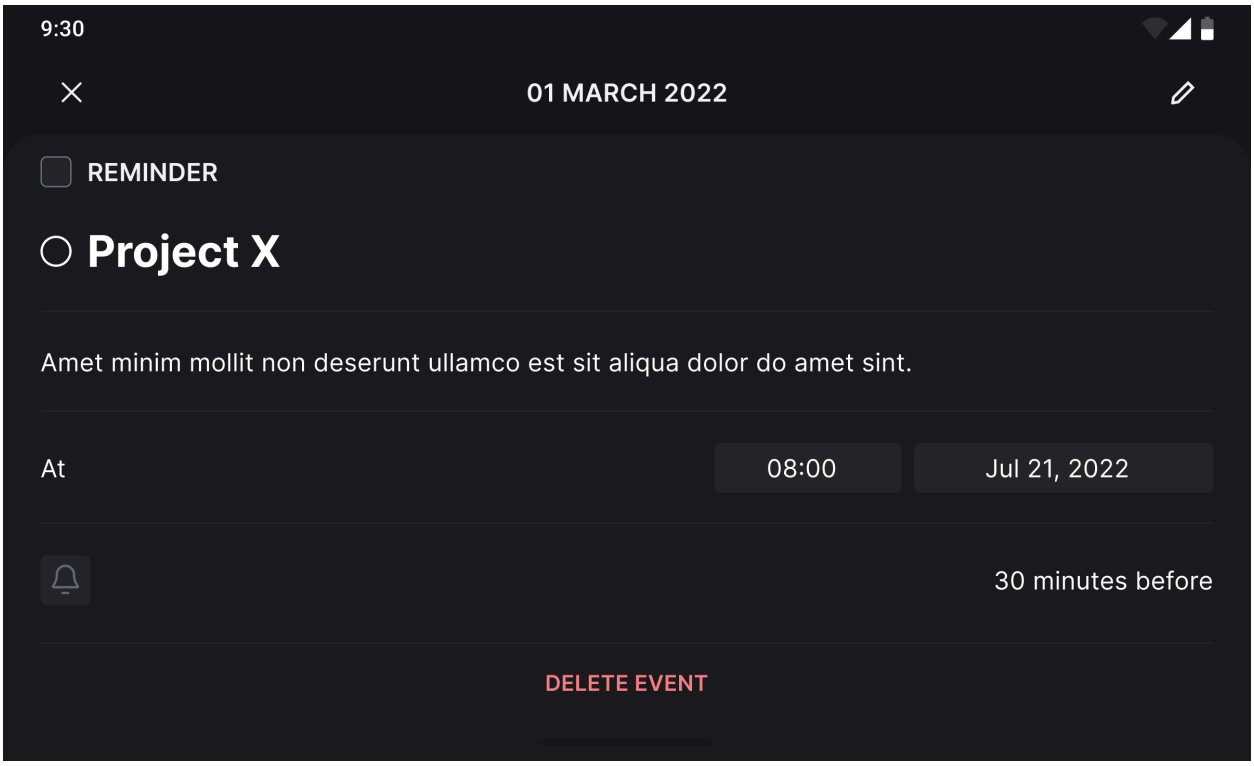
Reminder Detail - Edit Mode - Light - Portrait

Core Data:

- A reminder is the same as a task, just that you can't mark it as done/undone
- A reminder consists of the following data:
  - Title
  - Description
  - Time (Date + time)
  - Reminder time (same options as for events)

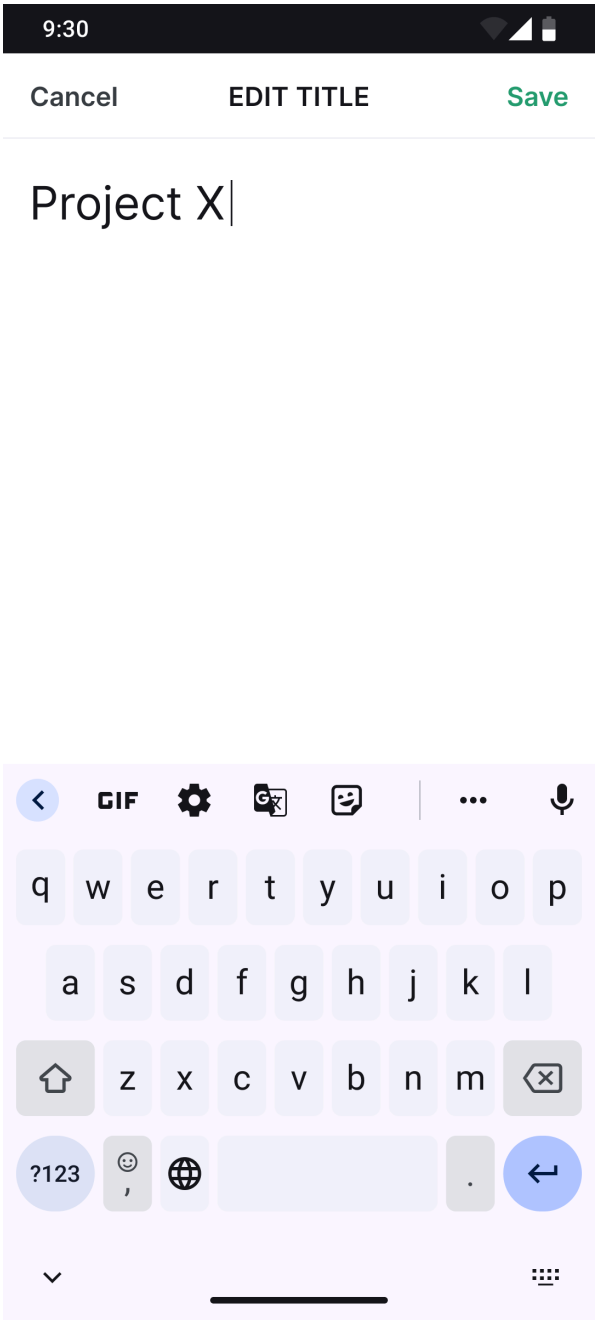
Edit Behavior:

- Same edit mechanism as events/tasks (view/edit modes)
- Title/description editing via separate screens
- Date/time selection via system dialogs
- Delete button shows confirmation bottomsheet

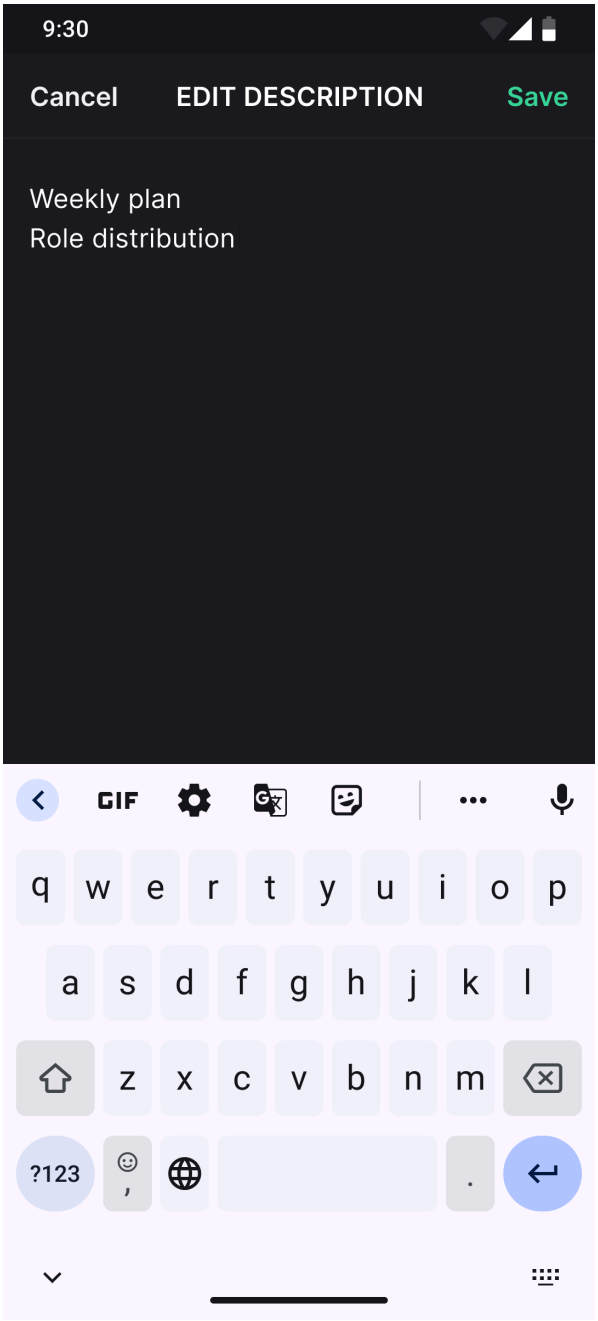


Reminder Detail - View Mode - Dark - Landscape

Edit Text Screen



Edit Title - Light - Portrait



Edit Description - Dark - Portrait

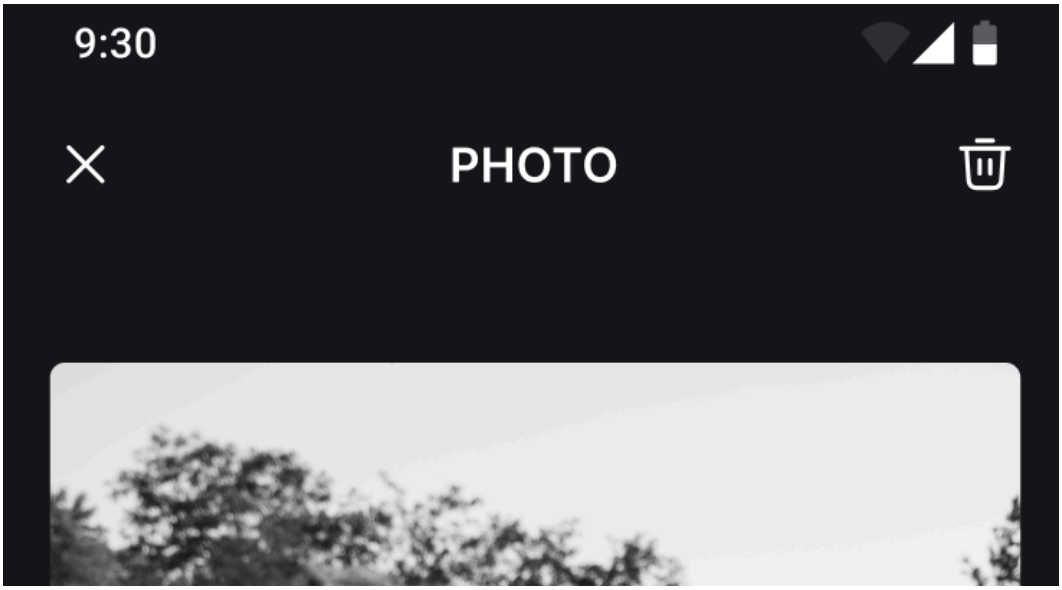
Dedicated text editing with responsive design:

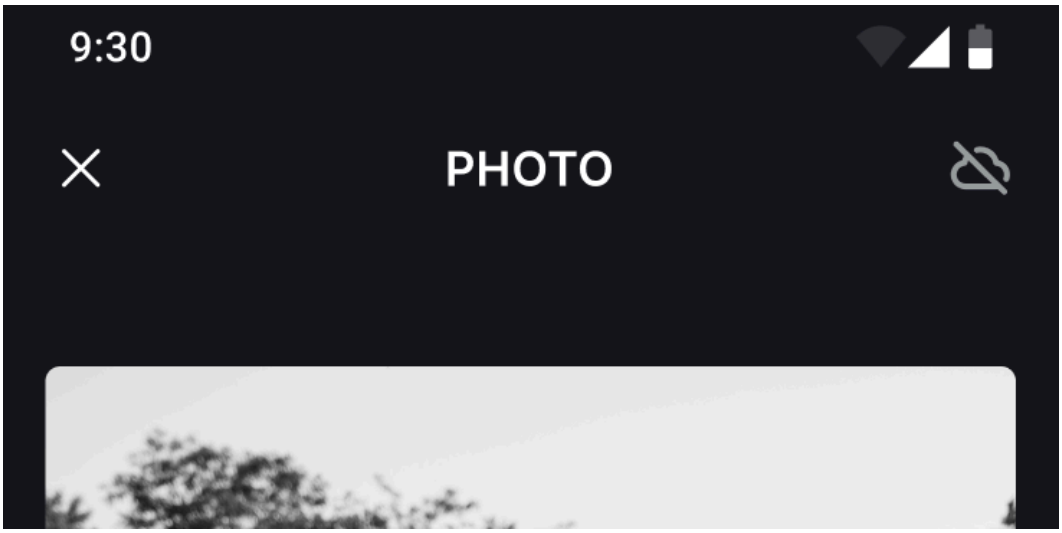
Functionality:

- Separate screen (not overlay) for title/description editing

- Changes NOT saved until parent detail screen save action
- Supports both single-line (title) and multi-line (description) editing

Photo Detail Screen





Functionality:

- Full-screen photo viewing
- Delete option for photo removal from event
    - Not available in offline-mode

    - Show indicator instead of trash icon when device has no internet connection

Notification Reminders

Core Functionality:

- Schedule reminders *(not to be confused with the reminder agenda item)* at selected reminder times
- Show notifications with agenda item title and description when reminder time reached
- Notification tap opens corresponding detail screen
- User-specific notifications only - only show notifications relevant for the logged in user

Cross-device Synchronization:

- Persist reminders across device restarts
- Handle agenda items added on other devices
- Check for new agenda items every 30 minutes minimum
- Set reminders for events where user is added as attendee

Smart Scheduling:

- Only show notifications if reminder time is in future
    - **Example:** The current time is 5pm. Event A lasts from 6pm-10pm and the reminder is set to 6h before. That notification shouldn't show up because 6pm - 6h = 12pm which is earlier than the current time.
- Clean up reminders when agenda items deleted
- Remove all reminders on user logout

Offline-first Architecture

Offline Capabilities:

- Full app functionality when offline (post-login)

- Add, edit, delete agenda items offline
- Automatic sync when connection restored
- Proper sync ordering for data consistency

Online-only Restrictions:

- Photo management (add/delete) requires internet connection
- Event attendee management requires internet connection
- Logging in & out requires internet connection and a successful REST request
- Offline indicators shown for restricted features

Sync Strategy:

- Use proper sync order to maintain backend consistency
- Handle conflicts with last-write-wins approach
- Cache remote photos for offline viewing
- Queue local changes for upload when online